



Python Interview Questions & Answers (120)

1. Python Basics (10 Qs)

Q1. What are Python's key features?

A: Interpreted, high-level, dynamically typed, object-oriented, cross-platform, supports multiple paradigms, and has extensive libraries.

Q2. What are Python's data types?

A: int, float, complex, str, list, tuple, set, dict, bool, NoneType.

Q3. What is PEP 8 and why is it important?

A: Python's style guide; ensures consistent, clean, and readable code.

Q4. Explain mutable vs immutable objects.

A: Mutable (list, dict, set) can change. Immutable (str, tuple, int) cannot.

Q5. What are Python's built-in data structures?

A: List, tuple, set, dictionary.

Q6. Difference between Python 2 and Python 3.

A: Python 3 uses Unicode strings by default, print() is a function, better division handling, etc.

Q7. What are Python's memory management features?

A: Private heap, reference counting, garbage collector.

Q8. Explain indentation in Python.

A: Indentation defines code blocks, mandatory unlike other languages.

Q9. How is Python interpreted?

A: Compiled into bytecode, executed by Python Virtual Machine (PVM).

Q10. Explain namespaces.

A: Mapping of variable names to objects (local, global, built-in).

2. Data Structures (15 Qs)

Q11. List vs Tuple differences.

A: List = mutable, slower. Tuple = immutable, faster, hashable.

Q12. How are sets used?

A: Store unique elements, fast membership tests.

Q13. What are dictionaries?

A: Key-value pairs, unordered, fast lookups.

Q14. How to merge dictionaries?

A: `{**dict1, **dict2}` or `dict1.update(dict2)`.

Q15. Remove duplicates from list?

A: `list(set(lst))` or `list(dict.fromkeys(lst))`.

Q16. What are list comprehensions?

A: Compact syntax for lists: `[x**2 for x in range(5)]`.

Q17. Flatten nested list?

A: `sum(nested, [])` or `itertools.chain(*nested)`.

Q18. Shallow vs deep copy?

A: Shallow copies refs; deep copy clones objects recursively.

Q19. How slicing works?

A: `list[start:end:step]` creates sublist.

Q20. Reverse a list?

A: `lst[::-1]` or `reversed(lst)`.

Q21. Frozenset in Python?

A: Immutable set, hashable, can be dict keys.

Q22. Difference between `is` and `==`?

A: `is` → identity, `==` → equality.

Q23. Sort dict by value?

A: `sorted(d.items(), key=lambda x: x[1])`.

Q24. Explain `heapq`.

A: Heap queue (priority queue) algorithms.

Q25. Implement stack & queue?

A: Stack → list `append/pop`. Queue → `collections.deque`.

3. Functions & OOP (20 Qs)

Q26. Function vs method?

A: Function = independent, method = bound to object.

****Q27. `*args` and `kwargs`?**

A: `*args` = variable positional args, `**kwargs` = variable keyword args.

Q28. Recursion example?

A: Function calling itself (factorial, Fibonacci).

Q29. What are decorators?

A: Functions that wrap others to extend behavior.

Q30. What are generators?

A: Functions with yield, return iterators, memory-efficient.

Q31. Iterator vs generator?

A: Iterator uses `__iter__`/`__next__`; generator auto creates them with yield.

Q32. With statement?

A: Ensures resource cleanup (e.g., file handling).

Q33. What are closures?

A: Function with access to variables from enclosing scope.

Q34. Lambda functions?

A: Small anonymous functions: `lambda x: x*2`.

Q35. `@staticmethod` vs `@classmethod`?

A: Staticmethod → no class/self ref. Classmethod → takes class as first arg.

Q36. Method Resolution Order (MRO)?

A: Defines method lookup order in inheritance hierarchy.

Q37. What is init?

A: Constructor method, initializes object attributes.

Q38. Inheritance in Python?

A: Reuse properties/methods from parent class.

Q39. Multiple vs multilevel inheritance?

A: Multiple → multiple parents. Multilevel → chain of inheritance.

Q40. What is polymorphism?

A: Same method name with different implementations.

Q41. Abstraction?

A: Hiding implementation, exposing only essential details.

Q42. Encapsulation?

A: Bundling data & methods; restricts direct access.

Q43. Magic methods?

A: Special methods like `__add__`, `__len__`, `__str__`.

Q44. str vs repr?

A: str → readable. repr → unambiguous (debugging).

Q45. Monkey patching?

A: Dynamically changing class/module behavior at runtime.

4. Exception Handling (10 Qs)

Q46. Errors vs exceptions?

A: Errors = compile/runtime issues. Exceptions = events handled via try-except.

Q47. Try-except-finally?

A: Try → code, Except → handle, Finally → always executes.

Q48. Raise vs assert?

A: Raise → throw exception. Assert → test condition.

Q49. Custom exceptions?

A: Subclass Exception to define custom error.

Q50. Exception chaining?

A: One exception caused by another (raise ... from ...).

Q51. Syntax vs runtime error?

A: Syntax → invalid code. Runtime → occurs during execution.

Q52. Catch multiple exceptions?

A: except (TypeError, ValueError):.

Q53. Use of else in exception handling?

A: Executes only if no exception in try block.

Q54. If exception not handled?

A: Program terminates with traceback.

Q55. Pass vs continue in exceptions?

A: Pass = do nothing. Continue = move to next iteration.

5. Modules & Packages (10 Qs)

Q56. Module vs package?

A: Module = single .py file. Package = collection with init.py.

Q57. Role of init.py?

A: Marks directory as package.

Q58. Install packages?

A: pip install package-name.

Q59. sys module use?

A: Access system-specific functions (args, paths).

Q60. Virtual environments?

A: Isolated Python environments.

Q61. Import dynamically?

A: `importlib.import_module()`.

Q62. What is pip?

A: Python package manager.

Q63. Import mechanism?

A: Searches `sys.path` → loads module → caches in `sys.modules`.

Q64. Absolute vs relative imports?

A: Absolute = full path. Relative = relative to current module.

Q65. `dir()` function?

A: Lists attributes/methods of object/module.

6. File Handling (10 Qs)

Q66. Read/write files?

A: `open(filename, 'r/w')`, use `read()/write()`.

Q67. `read()`, `readline()`, `readlines()`?

A: `read` → all, `readline` → one line, `readlines` → list of lines.

Q68. `with` statement in files?

A: Auto closes file.

Q69. Binary vs text files?

A: Binary = raw bytes. Text = human-readable.

Q70. Handle file exceptions?

A: `try-except IOError/OSerror`.

Q71. `tell()` and `seek()`?

A: `tell()` → current pointer. `seek()` → move pointer.

Q72. Check if file exists?

A: `os.path.exists(filename)`.

Q73. Delete file?

A: `os.remove(filename)`.

Q74. Copy/move files?

A: `shutil.copy()/shutil.move()`.

Q75. JSON handling?

A: `json.load()`, `json.dump()`.

7. Libraries for Data Analysis (15 Qs)

Q76. What is NumPy?

A: Numerical library with multi-dimensional arrays, fast computation.

Q77. NumPy vs Python list?

A: NumPy → fixed type, faster, vectorized.

Q78. Create NumPy array?

A: `np.array([1,2,3])`.

Q79. Broadcasting?

A: Automatic alignment of arrays of different shapes.

Q80. Handle missing values in Pandas?

A: `fillna()`, `dropna()`.

Q81. loc vs iloc?

A: loc → label-based. iloc → index-based.

Q82. Merge vs join?

A: merge = SQL-style. join = index-based.

Q83. Groupby in Pandas?

A: Splits data, applies function, combines results.

Q84. Pivot in Pandas?

A: Reshape data into table form.

Q85. concat vs append?

A: concat = multiple objects. append = one object.

Q86. Filter rows in Pandas?

A: `df[df['col'] > 10]`.

Q87. Vectorization in NumPy?

A: Apply operations directly on arrays, no loops.

Q88. Series vs DataFrame?

A: Series = 1D. DataFrame = 2D table.

Q89. Remove duplicates?

A: `df.drop_duplicates()`.

Q90. Sort values?

A: `df.sort_values('col')`.

8. Advanced Python (15 Qs)

Q91. What is GIL?

A: Global Interpreter Lock, allows one thread at a time.

Q92. Shallow vs deep copy?

A: Shallow → refs. Deep → clones.

Q93. Duck typing?

A: Behavior matters, not type.

Q94. Garbage collection?

A: Automated memory cleanup using reference counting.

Q95. Metaclasses?

A: Classes of classes; control class creation.

Q96. Reflection in Python?

A: Inspect objects at runtime (`getattr()`, `type()`).

Q97. Multithreading?

A: Run multiple threads in same process.

Q98. Multiprocessing?

A: Run parallel processes, bypasses GIL.

Q99. asyncio?

A: Asynchronous IO framework for concurrency.

Q100. Threading vs multiprocessing?

A: Threading shares memory, multiprocessing uses separate memory.

Q101. Coroutine?

A: Async function that can pause/resume execution.

Q102. Memory management?

A: Private heap, garbage collector.

Q103. Serialization in Python?

A: Convert objects to byte streams (pickle, json).

Q104. Pickle module?

A: Serialize/deserialize Python objects.

Q105. Monkey patching?

A: Modify class/module behavior dynamically.

9. Data Science-Specific Python (10 Qs)

Q106. Clean data in Python?

A: Handle missing, duplicates, outliers using Pandas.

Q107. Handle categorical variables?

A: One-hot encoding, label encoding.

Q108. One-hot encoding?

A: Convert categorical → binary vectors.

Q109. Normalize data?

A: Scale values to 0–1 using MinMaxScaler.

Q110. Feature scaling?

A: Standardization, normalization to equalize feature ranges.

Q111. train_test_split()?

A: Split dataset into train & test sets.

Q112. Cross-validation?

A: Splitting data into multiple folds for robust validation.

Q113. fit(), transform(), fit_transform()?

A: fit → learn. transform → apply. fit_transform → both.

Q114. Save/load models?

A: joblib.dump(), joblib.load().

Q115. Evaluate regression model?

A: Metrics: RMSE, MAE, R^2 .

10. Miscellaneous (5 Qs)

Q116. datetime module?

A: Date/time handling, parsing, formatting.

Q117. Parse XML/HTML?

A: Use BeautifulSoup, xml.etree.ElementTree.

Q118. Regex in Python?

A: re module for pattern matching.

Q119. itertools module?

A: Tools for iteration (permutations, combinations).

Q120. Send HTTP requests?

A: requests library: requests.get(url).